Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL604**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**05**
Title of the Experiment:	**Implementation of Hebbian Learning Algorithms**
Date of Performance:	**12/02/2026**
Date of Submission:	**26/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs. Sneha Yadav

Signature :

Date:

Aim: Implementation of Hebbian Learning Algorithms.

Objective: Able to interpret the implementation of Hebbian Learning Algorithms.

Theory:

Hebbian learning is a foundational concept in neuroscience and artificial neural networks. It describes how the strength of connections (weights) between neurons is adjusted based on their activity. This learning mechanism is often summarized as:
"Neurons that fire together, wire together."

Historical Context
The Hebbian learning rule was first proposed by Donald Hebb in his 1949 book "The Organization of Behavior". It was an attempt to explain how neurons adapt during the learning process and how memory and associations are formed in the brain.

Core Concepts of Hebbian Learning

Synaptic Plasticity:
The strength of the synaptic connection between two neurons is modified based on their simultaneous activation.
If both the pre-synaptic and post-synaptic neurons are active simultaneously, the connection strengthens.

Locality:
The weight adjustment depends only on the states of the connected neurons, without external supervision or global feedback.

Unsupervised Learning:
Hebbian learning does not require labeled data or error correction. Instead, it adjusts weights based solely on the correlation of neuronal activity.

Variants of Hebbian Learning - -

Standard Hebbian Learning:

The weight change is proportional to the product of the activations of the pre- and post-synaptic neurons.

Oja's Rule (Stabilized Hebbian Learning):

To prevent unbounded weight growth, Oja introduced a normalization term.

Covariance Rule:

Instead of directly using activations, this rule subtracts the mean activation

Hebbian Anti-Hebbian Learning:

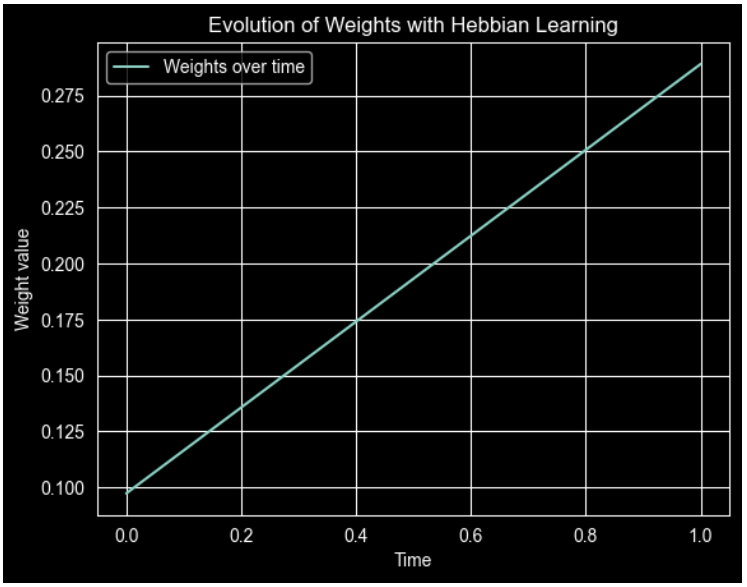
Positive correlations increase weights (Hebbian rule), while negative correlations decrease weights (anti-Hebbian rule)

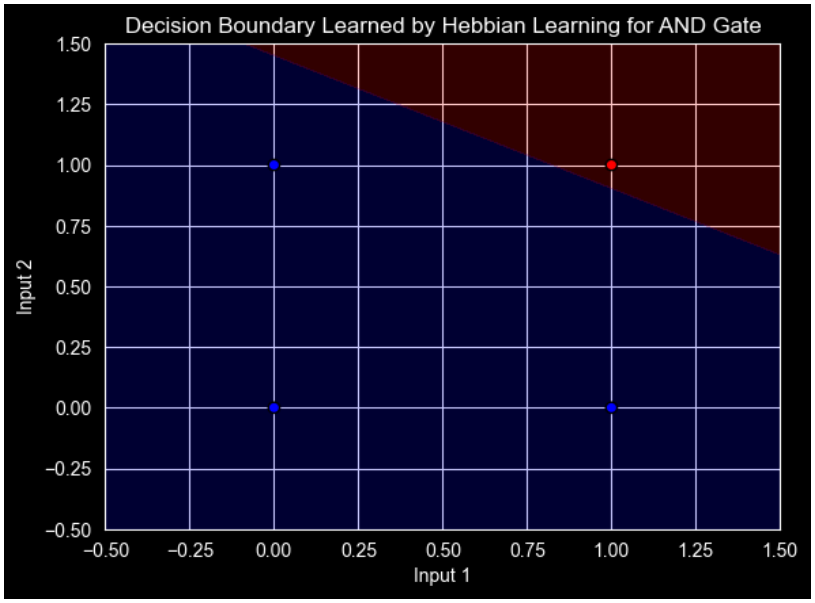
Implementation:

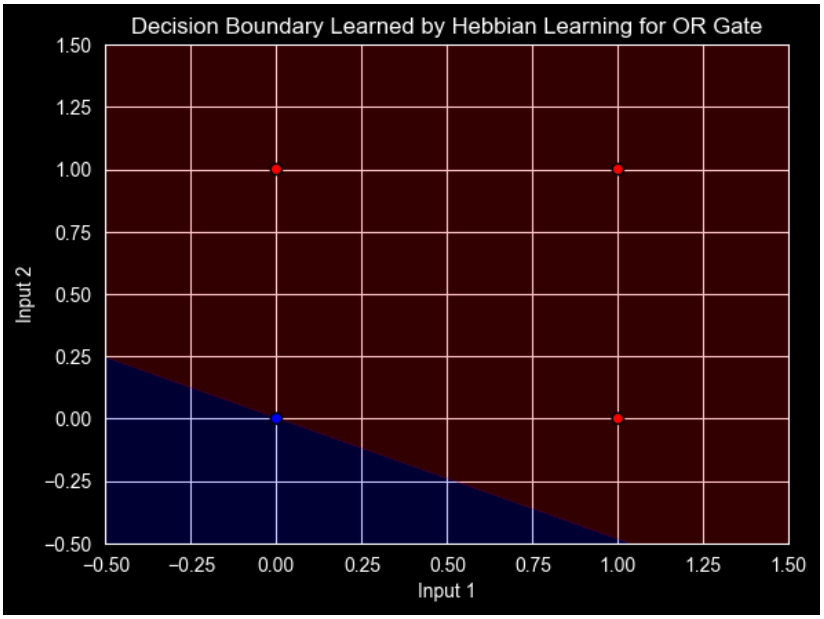
```
import numpy as np
import matplotlib.pyplot as plt


n_inputs = 2
weights = np.random.rand(n_inputs)

def hebbian_update(inputs, output, weights, learning_rate=0.01):
      for i in range(len(weights)):
       weights[i] += learning_rate * inputs[i] * output


inputs = np.array([[0, 1], [1, 0], [1, 1]])
outputs = np.array([1, 1, 2])

for input, output in zip(inputs, outputs):
   hebbian_update(input, output, weights)
   print("Updated weights:", weights)


plt.plot(weights, label='Weights over time')
plt.xlabel('Time')
```

```
plt.ylabel('Weight value')
plt.title('Evolution of Weights with Hebbian Learning')
plt.legend()
plt.show()

inputs = np.array([[1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
outputs = np.array([0, 0, 0, 1])


def hebbian_learning(inputs, outputs, learning_rate=0.1, epochs=20):
    num_inputs = inputs.shape[1]
    weights = np.random.rand(num_inputs)  # Initialize weights randomly

      for epoch in range(epochs):
       for input, output in zip(inputs, outputs):
                neuron_output = 1 if np.dot(weights, input) > 0 else 0
                weights += learning_rate * input * (output - neuron_output)
return weights

inputs = np.array([[1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
outputs = np.array([0, 0, 0, 1])

weights = hebbian_learning(inputs, outputs)
print("Learned weights:", weights)

x = np.linspace(-0.5, 1.5, 100)
y = np.linspace(-0.5, 1.5, 100)
X, Y = np.meshgrid(x, y)
Z = weights[0] + weights[1] * X + weights[2] * Y

plt.contourf(X, Y, Z, levels=[-100, 0, 100], alpha=0.2, colors=['blue', 'red'])
plt.scatter(inputs[:, 1], inputs[:, 2], c=outputs, cmap='bwr', edgecolor='k')

plt.title('Decision Boundary Learned by Hebbian Learning for AND Gate')
plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.grid(True)
plt.show()


import numpy as np
import matplotlib.pyplot as plt

inputs = np.array([
    [1, 0, 0],
    [1, 0, 1],
    [1, 1, 0],
    [1, 1, 1]
```

```
])

outputs = np.array([0, 1, 1, 1])


def hebbian_learning(inputs, outputs, learning_rate=0.1, epochs=20):
    num_inputs = inputs.shape[1]
    weights = np.random.rand(num_inputs)  # Initialize weights randomly

    for epoch in range(epochs):
        for input, output in zip(inputs, outputs):
            neuron_output = 1 if np.dot(weights, input) > 0 else 0
            weights += learning_rate * input * (output - neuron_output)

    return weights

weights = hebbian_learning(inputs, outputs)
print("Learned weights:", weights)

x = np.linspace(-0.5, 1.5, 100)
y = np.linspace(-0.5, 1.5, 100)
X, Y = np.meshgrid(x, y)
Z = weights[0] + weights[1] * X + weights[2] * Y

plt.contourf(X, Y, Z, levels=[-100, 0, 100], alpha=0.2, colors=['blue', 'red'])
plt.scatter(inputs[:, 1], inputs[:, 2], c=outputs, cmap='bwr', edgecolor='k')

plt.title('Decision Boundary Learned by Hebbian Learning for OR Gate')
plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.grid(True)
plt.show()
```







Conclusion:

Comment on understanding of Hebbian Learning Algorithm and how it is implemented.

Hebbian learning is a neuroscience-based principle often summarized as "cells that fire together, wire together," where the weight of a connection between two neurons increases when they are activated simultaneously.